

DAFTAR TECHNOLOGY YANG DIGUNAKAN

Daftar technology yang dipakai pada aplikasi perpustakaan berbasis microservice, beserta alasan pemilihannya:

Ringkasan

Kategori	Technology	Fungsi Singkat
Frontend	HTML5 & CSS3	Struktur dan tampilan halaman katalog serta peminjaman
Frontend	Vanilla JavaScript & Fetch API	Logika halaman dan pengambilan data dari backend
Backend	Node.js	Runtime untuk menjalankan semua service
Backend	Express.js	Framework pembuat REST API
Backend	CORS	Mengizinkan frontend mengakses backend
Arsitektur	Microservice	Memisahkan Book Service dan Borrowing Service
Komunikasi	REST API (JSON over HTTP)	Pertukaran data antarservice dan frontend
Tools	Git & GitHub	Version control dan kolaborasi kelompok
AI Coding Tool	Gemini 3.1 & ChatGPT-5.6 Luna	Membantu pengembangan dan debugging kode

1. Arsitektur Microservice

Aplikasi dipecah menjadi dua service yang berjalan mandiri:

Service	Port	Tanggung Jawab
Book Service	5001	Mengelola data buku (katalog, detail, ketersediaan)
Borrowing Service	5002	Mengelola proses peminjaman buku

Dipilih karena:

- Setiap service mempunyai satu fungsi yang jelas, sehingga kode lebih mudah dipahami dan dirawat.
- Perubahan di satu service tidak langsung merusak service lain.
- Anggota kelompok bisa mengerjakan service yang berbeda secara paralel tanpa saling bentrok.
- Sesuai dengan ketentuan tugas: minimal 2 service yang berkomunikasi lewat API.

2. Node.js

Node.js adalah runtime JavaScript yang berjalan di sisi server. Digunakan untuk menjalankan Book Service dan Borrowing Service.

Dipilih karena:

- Frontend dan backend sama-sama memakai JavaScript, jadi anggota kelompok tidak perlu mempelajari banyak bahasa.

- Ringan dan cepat dijalankan, cocok untuk service kecil yang dijalankan di beberapa port.
- Ekosistem package (npm) sangat lengkap.

3. Express.js

Express.js adalah framework minimalis untuk Node.js yang digunakan untuk membuat server dan REST API. Dipakai untuk mendefinisikan endpoint di tiap service (misalnya GET, POST) dan mengatur request serta response.

Dipilih karena:

- Sederhana dan mudah dipelajari, cocok untuk proyek kuliah dengan waktu terbatas.
- Struktur route yang jelas, sehingga mudah diperiksa dan diperbaiki setelah kode dibuat AI.
- Dokumentasi dan contoh penggunaannya banyak.

4. CORS

CORS adalah package middleware untuk mengatur Cross-Origin Resource Sharing. Digunakan untuk mengizinkan halaman frontend memanggil API di Port 5001 dan 5002.

Dipilih karena:

- Browser secara default memblokir request antar-origin (misalnya halaman di satu alamat memanggil API di port berbeda).
- Tanpa CORS, frontend tidak bisa mengambil data dari backend meskipun server sudah berjalan.

5. REST API dan JSON

REST API dan JSON adalah gaya penulisan API berbasis HTTP, dengan data dikirim dalam format JSON. Dipakai untuk komunikasi frontend ke service, dan antarservice.

Dipilih karena:

- Memenuhi ketentuan tugas bahwa antarservice harus berkomunikasi menggunakan API.
- JSON ringan, mudah dibaca manusia, dan didukung langsung oleh JavaScript.
- Mudah diuji dengan tools seperti Postman atau langsung dari browser.

Daftar endpoint:

Service	Method & Endpoint	Fungsi
Book Service (5001)	GET /api/books	Mengambil semua daftar buku
Book Service (5001)	GET /api/books/:id	Mengambil detail satu buku
Book Service (5001)	PUT /api/books/:id/status	Memperbarui status ketersediaan buku (dipanggil Borrowing Service)
Borrowing Service (5002)	GET /api/borrowings/:nim	Mengambil peminjaman aktif milik satu mahasiswa

Service	Method & Endpoint	Fungsi
Borrowing Service (5002)	POST /api/borrowings	Memproses peminjaman (maksimal 3 buku aktif per NIM, jatuh tempo 7 hari)

6. Komunikasi Antarservice

Borrowing Service memanggil Book Service melalui HTTP pada saat proses peminjaman, dengan dua langkah:

Langkah	Request ke Book Service	Tujuan
1	GET /api/books/:id	Memeriksa apakah buku ada di katalog dan masih tersedia (isAvailable)
2	PUT /api/books/:id/status	Mengubah status buku menjadi tidak tersedia setelah peminjaman dicatat

Library yang digunakan: **fetch bawaan Node.js**.

Jika Book Service tidak dapat dihubungi, Borrowing Service mengembalikan respons error 500 dengan pesan "Gagal terhubung ke Book Service".

Dipilih karena:

- Setiap service tetap independen: Borrowing Service tidak mengakses data Book Service secara langsung, melainkan lewat API-nya.
- Cara ini menunjukkan prinsip microservice, yaitu service saling berinteraksi hanya lewat antarmuka yang jelas.

7. Frontend: HTML5, CSS3, JavaScript Vanilla, dan Fetch API

Technology	Dipakai untuk
HTML5	Struktur halaman katalog dan peminjaman
CSS3	Tampilan dan tata letak
JavaScript Vanilla	Logika interaksi di halaman
Fetch API	Mengirim dan mengambil data dari backend tanpa reload halaman

Frontend terdiri dari dua halaman, yaitu index.html (katalog buku) dan peminjaman.html (peminjaman buku). Tampilan diatur oleh style.css, logika dan pemanggilan API oleh script.js, dan folder images/ berisi aset gambar (gambar1.jpg sampai gambar10.jpg).

Dipilih karena:

- Fokus tugas ada pada arsitektur microservice, bukan frontend. Tanpa framework tambahan, tampilan tetap ringan dan tidak butuh proses build.
- Fetch API sudah tersedia di browser, jadi tidak perlu library tambahan.

- Mudah dijalankan dan dipahami semua anggota kelompok, termasuk yang belum familiar dengan framework frontend.

8. Git dan GitHub

Dipakai untuk menyimpan source code, mencatat kontribusi tiap anggota, dan mengumpulkan tugas.

Dipilih karena:

- Wajib sesuai ketentuan pengumpulan (repository GitHub Public, satu repository per kelompok).
- Riwayat commit menjadi bukti kontribusi setiap anggota.
- Memudahkan kerja bersama tanpa saling menimpa file.

9. AI Coding Tool: Gemini 3.1 dan ChatGPT-5.6 Luna

Digunakan untuk membantu membuat kode server, menyesuaikan fungsi, dan memperbaiki error.

Dipilih karena:

- Mempercepat pembuatan kerangka kode (boilerplate) dan endpoint.
- Membantu menjelaskan error saat debugging.
- Menggunakan dua tool memberi perbandingan jawaban, sehingga hasil bisa dicek silang.

10. Cara Menjalankan (Ringkas)

Jalankan dari folder yang berisi book-service, borrowing-service, dan frontend.

```
# Book Service
cd book-service
npm install
node server.js      # berjalan di http://localhost:5001

# Borrowing Service (buka terminal baru)
cd borrowing-service
npm install
node server.js      # berjalan di http://localhost:5002
```

Setelah kedua service berjalan, buka frontend/index.html di browser.

MASALAH ATAU KESALAHAN YANG DITEMUKAN DARI HASIL AI

Setelah aplikasi perpustakaan hasil generate AI diuji, ditemukan dua kesalahan.

1. Status buku tidak kembali menjadi "Tersedia" setelah dikembalikan

- **Langkah Pengujian:** Pengguna meminjam sebuah buku, lalu buku tersebut dikembalikan.
- **Hasil yang diharapkan:** Saat dipinjam, status buku berubah menjadi “Tidak Tersedia”. Setelah dikembalikan, status kembali menjadi “Tersedia”.
- **Hasil yang terjadi:** Status berubah menjadi “Tidak Tersedia” saat dipinjam dengan benar, tetapi tetap “Tidak Tersedia” setelah buku dikembalikan.
- **Dampak:** Buku yang sudah dikembalikan tidak bisa dipinjam lagi oleh pengguna lain, sehingga data ketersediaan buku menjadi tidak akurat.
- **Perbaikan:** Ditemukan bahwa fungsi pengembalian buku di `script.js` hanya mencari buku dan menyimpan ulang daftar buku (`setBooks(books)`) tanpa mengubah statusnya, sehingga `isAvailable` tetap `false`. Ditambahkan satu baris `books[bookIndex].isAvailable = true;` sebelum data disimpan, sehingga buku yang dikembalikan kembali berstatus "Tersedia" dan dapat dipinjam pengguna lain. Selanjutnya pada versi Microservice, logika ini disesuaikan dengan mengirimkan request API `PUT /api/books/:id/status` ke Book Service untuk memperbarui status ketersediaan buku.

```
385 const books = getBooks();
386 const bookIndex = books.findIndex((b) => b.id === bookId);
387 if (bookIndex !== -1) {
388   setBooks(books);
389 }
```

Gambar 1: kode yang salah

```
385 const books = getBooks();
386 const bookIndex = books.findIndex((b) => b.id === bookId);
387 if (bookIndex !== -1) {
388   books[bookIndex].isAvailable = true;
389   setBooks(books);
390 }
```

Gambar 2: kode yang benar

2. Fitur pencarian tidak memfilter katalog buku

- **Langkah Pengujian:** Pengguna membuka web di browser, lalu mengetik judul buku pada kolom pencarian.
- **Hasil yang diharapkan:** Daftar katalog langsung menyaring dan hanya menampilkan buku yang sesuai dengan judul yang diketik.
- **Hasil yang terjadi:** Tampilan katalog tidak berubah dan semua buku tetap ditampilkan tanpa proses penyaringan.
- **Dampak:** Kolom pencarian tidak berfungsi, sehingga pengguna sulit menemukan buku tertentu, terutama jika jumlah buku banyak.
- **Perbaikan:** Ditemukan bahwa id kolom pencarian pada file HTML (`search-box-input`) tidak sama dengan id yang dicari oleh `script.js` (`search-input`), sehingga JavaScript tidak menemukan elemennya dan fungsi penyaringan tidak berjalan. Id pada HTML diubah menjadi `search-input` agar sesuai dengan `script.js`, sehingga katalog kini tersaring sesuai judul yang diketik.

```

76 <!-- SEARCH BAR -->
77 <div class="search-box">
78   <input
79     type="text"
80     id="search-box-input"
81     placeholder="Cari judul atau pengarang..."

```

Gambar 3: kode yang salah

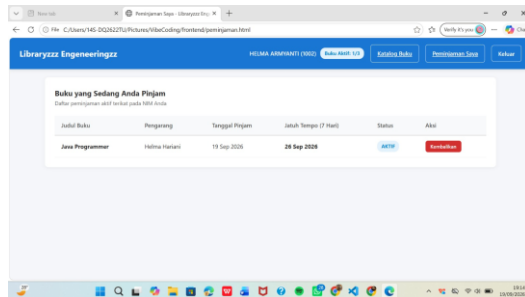
```

76 <!-- SEARCH BAR -->
77 <div class="search-box">
78   <input
79     type="text"
80     id="search-input"
81     placeholder="Cari judul atau pengarang..."

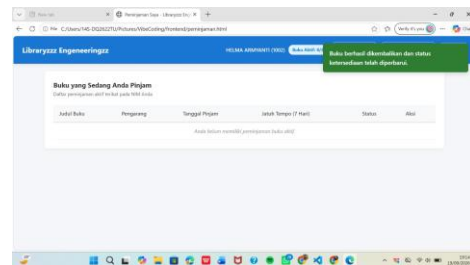
```

Gambar 4: kode yang benar

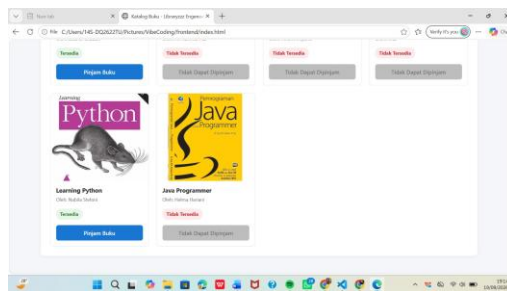
• Bukti Tampilan Eror 1



Gambar 5: buku yang dipinjam

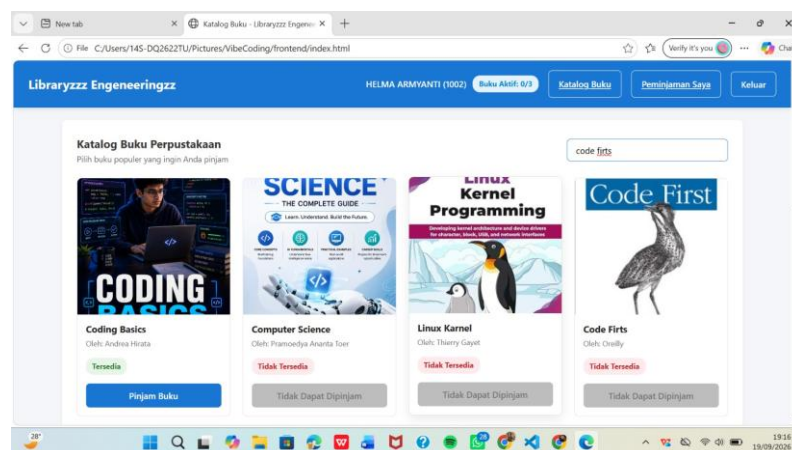


Gambar 6: buku dikembalikan



Gambar 7: buku yang sudah dikembalikan tetap tidak tersedia

• Bukti Tampilan Eror 2



Gambar 8: Pencarian tidak memfilter katalog buku